



UNIVERSITÀ DI TRENTO
DIPARTIMENTO DI INGEGNERIA INDUSTRIALE
CORSO DI LAUREA IN INGEGNERIA INDUSTRIALE

~ · ~

ANNO ACCADEMICO 2023–2024

Confronto di approcci per il controllo ottimo della traiettoria di robot

Relatore
Prof. Andrea DEL PRETE

Studente laureando
Giacomo MONTAGNA
227753

APPELLO DI PROVA FINALE: 20 settembre 2024

Sommario

L'elaborato si propone di confrontare l'efficienza computazionale di due approcci per l'ottimizzazione della traiettoria di un robot manipolatore utilizzando l'algoritmo Ricorsivo di Newton-Eulero (RNEA) per la risoluzione della dinamica inversa o una combinazione di RNEA con l'Algoritmo del Corpo Rigido Composito (CRBA).

Il problema è stato implementato in ambiente Python, fornendo prima le formulazioni teoriche necessarie. Per la realizzazione degli algoritmi della dinamica inversa è stata ampliata la libreria ADAM. I risultati mostrano quale dei due approcci sia più efficiente in termini di tempo di calcolo, evidenziando l'importanza della scelta dell'algoritmo nel contesto dell'ottimizzazione della traiettoria robotica.

Indice

Elenco dei simboli	v
Introduzione	1
1 Algebra spaziale	2
1.1 Introduzione	2
1.2 Spazi Vettoriali	2
1.3 Basi Duali	3
1.4 Velocità Spaziale	3
1.5 Accelerazione Spaziale	3
1.6 Forza Spaziale	3
1.7 Inerzia Spaziale	4
1.8 Prodotti Vettoriali Spaziali	4
1.9 Differenziazione in Coordinate Mobili	4
1.10 Cambiamento di coordinate	5
2 Modellazione di Robot	6
2.1 Topologia del sistema	6
2.2 Albero Cinematico	6
2.3 Vincoli di Movimento	6
2.4 Trasformazioni del Giunto ($\mathbf{X}_J$)	7
2.5 Trasformazioni dell'Albero ($\mathbf{X}_T$)	7
2.6 Il robot ur5	7
3 Algoritmi per la dinamica inversa	9
3.1 RNEA	9
3.2 CRBA	12
3.3 Implementazione degli algoritmi con libreria ADAM	13
4 Problema di ottimizzazione della traiettoria	15
4.1 Introduzione	15
4.2 Variabili del Problema	15
4.3 Equazioni di Stato	15
4.4 Funzione di Costo	16
4.5 Vincoli del Problema	16
4.6 Risoluzione del Problema con CasADi	17
4.7 Confronto tra gli Approcci	18

INDICE

Conclusioni	20
Bibliografia	21
Elenco delle figure	22
Elenco delle tabelle	23
Ringraziamenti	24

Elenco dei simboli

stile grassetto I Vettori e le matrici vengono indicati in stile grassetto. In caso di dubbio tra vettore spaziale e 3D, quello spaziale è indicato come $\hat{\mathbf{v}}$ mentre quello 3D come $\underline{\mathbf{v}}$.

$\mathbf{v}$	Velocità, può essere spaziale o lineare
$\boldsymbol{\omega}$	Velocità angolare
$\mathbf{a}$	Accelerazione, può essere spaziale o 3D
$\mathbf{f}$	Forza, può essere spaziale o lineare
$\mathbf{n}$	Coppia
$\mathbf{v}_i$	Velocità del corpo i
$\mathbf{a}_i$	Accelerazione del corpo i
$\mathbf{1}$	Matrice identità
$\mathbf{0}$	Matrice nulla
$\mathbf{f}_i^B$	Forza netta che agisce sul corpo i
$\mathbf{f}_i^x$	Forza esterna che agisce sul corpo i
$\mathbf{I}_i^c$	Inerzia del sottosistema radicato nel corpo i , trattata come un singolo corpo rigido composito
$\mathbf{I}$	Matrice di inerzia
$\boldsymbol{\tau}$	Vettore delle forze applicate
$\mathbf{q}$	Posizioni dei giunti
$\dot{\mathbf{q}}$	Velocità dei giunti
$\ddot{\mathbf{q}}$	Accelerazioni dei giunti
$\mathbf{X}_J$	Matrice di cambio di coordinate del giunto
$\mathbf{X}_T$	Matrice di trasformazione dell'albero
$\mathbf{H}$	Matrice di inerzia del sistema
$\mathbf{C}$	Forze di Coriolis, centrifughe e gravitazionali

$\mathbf{S}$ Matrice del sottospazio di moto dei giunti

$\lambda(i)$ Indice del corpo genitore di i

$\mu(i)$ Insieme degli indici dei corpi figli di i

RNEA Algoritmo Ricorsivo di Newton-Eulero

CRBA Algoritmo del Corpo Rigido Composito

Introduzione

Questa tesi si propone di esaminare e confrontare due approcci distinti per la risoluzione del problema di ottimizzazione della traiettoria di un robot manipolatore, con particolare attenzione all'efficienza computazionale. In particolare, l'obiettivo è investigare se l'utilizzo esclusivo dell'algoritmo Ricorsivo di Newton-Eulero (RNEA) offra un vantaggio in termini di tempo di calcolo rispetto a un approccio combinato che include anche l'Algoritmo del Corpo Rigido Composito (CRBA).

I primi tre capitoli di questa tesi forniscono le basi teoriche necessarie per comprendere appieno il contesto e i metodi utilizzati nel quarto capitolo. Si inizia con una trattazione dell'algebra spaziale, che è fondamentale per la rappresentazione e il calcolo delle dinamiche dei corpi rigidi. Successivamente, viene esposta la modellazione dei robot, seguita dalla descrizione degli algoritmi di dinamica inversa, in particolare il RNEA e il CRBA, che sono essenziali per l'ottimizzazione delle traiettorie.

Nel quarto capitolo, le conoscenze teoriche acquisite vengono applicate all'ottimizzazione della traiettoria di un robot manipolatore. Il lavoro svolto prevede l'implementazione di un risolutore per il problema di controllo ottimo della traiettoria in ambiente Python, utilizzando la libreria CasADi. Questa libreria consente di formulare il problema tenendo conto delle dinamiche del sistema e dei vincoli imposti. Inoltre, è stata utilizzata la libreria ADAM, che implementa una raccolta di algoritmi per il calcolo della dinamica dei robot. Tuttavia, questa è stata modificata per completare l'implementazione dell'algoritmo RNEA.

L'analisi condotta mira a valutare quale dei due approcci considerati sia più efficiente in termini di tempo di calcolo.

Capitolo 1

Algebra spaziale

Il seguente capitolo e i capitoli 2 e 3 pongono le fondamenta teoriche per la comprensione del problema di ottimizzazione della traiettoria, i riferimenti principali sono i libri "Rigid Body Dynamics Algorithms" [3] e "Springer Handbook of Robotics" [5]

1.1 Introduzione

I vettori spaziali sono vettori 6D che combinano gli aspetti lineari e angolari dei moti e delle forze dei corpi rigidi. Essi forniscono una notazione per lo studio della dinamica dei corpi rigidi. La notazione vettoriale spaziale ci permette di sviluppare rapidamente le equazioni del moto e di esprimerle sinteticamente in forma simbolica. Ci permette anche di ricavare rapidamente algoritmi di dinamica e di esprimerli in una forma compatta, facilmente convertibile in codice informatico efficiente.

Di seguito verranno presentati i concetti principali dell'algebra spaziale necessari per la comprensione degli algoritmi della dinamica inversa.

1.2 Spazi Vettoriali

Per la descrizione dei vettori spaziali è necessario introdurre due spazi vettoriali:

- M^6 : Questo è lo spazio vettoriale che rappresenta i vettori di moto spaziali, che descrivono attributi di movimento come velocità e accelerazione.
- F^6 : Questo è lo spazio vettoriale che rappresenta i vettori di forza spaziali, che descrivono le forze agenti su un corpo rigido, inclusi impulso e momento.

Tra loro è definito un prodotto scalare tale che:

$$\hat{\mathbf{m}} \cdot \hat{\mathbf{f}} = \text{lavoro}$$
$$". " : \mathbb{M}^6 \times \mathbb{F}^6 \rightarrow \mathbb{R}$$

1.3 Basi Duali

Se $\{d_1, \dots, d_6\}$ è una base arbitraria su M^6 allora esiste una base reciproca unica $\{e_1, \dots, e_6\}$ su F^6 che soddisfa

$$d_i \cdot e_j = \begin{cases} 0 & : i \neq j \\ 1 & : i = j \end{cases}$$

Con queste basi, il prodotto scalare di due vettori spaziali è

$$\hat{m} \cdot \hat{f} = \hat{m}^T \hat{f}$$

1.4 Velocità Spaziale

Dato un corpo rigido B ed un punto O la cui posizione non cambia rispetto al sistema di riferimento solidale con il corpo (è quindi un "body-fixed point"), la velocità di un punto $P \in B$ può essere calcolata come:

$$\underline{v}_P = \underline{v}_O + \underline{\omega} \times \overrightarrow{OP}$$

dove $\underline{v}_O$ è la velocità lineare del punto O e $\underline{\omega}$ è la velocità angolare. La velocità di ogni punto del corpo può quindi essere descritta attraverso $\underline{v}_O$ ed $\underline{\omega}$.

Il vettore di velocità spaziale $\hat{v}$ è perciò definito come:

$$\hat{v} = \begin{bmatrix} \underline{\omega} \\ \underline{v}_O \end{bmatrix}$$

Con

$$\hat{v} = \omega_x d_{Ox} + \omega_y d_{Oy} + \omega_z d_{Oz} + v_{Ox} d_x + v_{Oy} d_y + v_{Oz} d_z.$$

1.5 Accelerazione Spaziale

L'accelerazione spaziale è la derivata temporale della velocità spaziale :

$$\hat{a} = \dot{\hat{v}} = \begin{bmatrix} \dot{\underline{\omega}} \\ \dot{\underline{v}}_O \end{bmatrix}$$

Seguendo le definizioni date per la velocità spaziale $\underline{v}_O(t)$ è la velocità alla quale i punti fissi rispetto al corpo fluiscono attraverso O .

$\dot{\underline{v}}_O$ è pertanto il tasso di variazione della velocità di flusso.

1.6 Forza Spaziale

Analogamente a quanto visto con la velocità la forza spaziale combina forze lineari e momenti torcenti:

$$\hat{f} = \begin{bmatrix} \underline{n}_O \\ \underline{f} \end{bmatrix}$$

dove $\underline{n}_O$ è il momento torcente (coppia) rispetto a O e $\underline{f}$ è la forza lineare.

Per un punto $P \in B$ il momento torcente $\underline{n}_P$ può essere calcolato come:

$$\underline{n}_P = \underline{n}_O + \underline{f} \times \overrightarrow{OP}$$

1.7 Inerzia Spaziale

Il tensore di inerzia spaziale di un corpo rigido definisce la relazione tra la sua velocità e il momento. L'inerzia spaziale è quindi una mappatura da M^6 a F^6 . Se un corpo rigido ha una velocità spaziale di $\mathbf{v}$ e un'inerzia spaziale $\mathbf{I}$, allora il suo momento è dato dalla formula

$$\mathbf{h} = \mathbf{I}\mathbf{v}$$

L'inerzia spaziale di un corpo rigido rispetto al centro di massa C è rappresentata da una matrice simmetrica 6×6 . La forma generale della matrice di inerzia spaziale $\mathbf{I}_C$ è data da:

$$\mathbf{I}_C = \begin{bmatrix} \bar{\mathbf{I}}_C & \mathbf{0} \\ \mathbf{0} & m\mathbf{1} \end{bmatrix}$$

dove:

- $\bar{\mathbf{I}}_C$ è il tensore di inerzia rotazionale 3×3 rispetto al centro di massa.
- m è la massa del corpo.
- $\mathbf{1}$ è la matrice identità 3×3 .

1.8 Prodotti Vettoriali Spaziali

Vengono definite due operazioni di prodotto vettoriale: una per i vettori di moto e una per le forze.

$$\begin{aligned} \hat{\mathbf{v}} \times \hat{\mathbf{m}} &= \begin{pmatrix} \boldsymbol{\omega} \\ \mathbf{v}_O \end{pmatrix} \times \begin{pmatrix} \mathbf{m} \\ \mathbf{m}_O \end{pmatrix} = \begin{pmatrix} \boldsymbol{\omega} \times \mathbf{m} \\ \boldsymbol{\omega} \times \mathbf{m}_O + \mathbf{v}_O \times \mathbf{m} \end{pmatrix} \\ \hat{\mathbf{v}} \times^* \hat{\mathbf{f}} &= \begin{pmatrix} \boldsymbol{\omega} \\ \mathbf{v}_O \end{pmatrix} \times^* \begin{pmatrix} \mathbf{n}_O \\ \mathbf{f} \end{pmatrix} = \begin{pmatrix} \boldsymbol{\omega} \times \mathbf{n}_O + \mathbf{v}_O \times \mathbf{f} \\ \boldsymbol{\omega} \times \mathbf{f} \end{pmatrix} \end{aligned}$$

dove $\hat{\mathbf{v}}$ e $\hat{\mathbf{m}}$ sono vettori di moto, e $\hat{\mathbf{f}}$ è un vettore di forza.

1.9 Differenziazione in Coordinate Mobili

Dato un sistema di coordinate A associato a un sistema di riferimento in movimento con una velocità spaziale $\mathbf{v}_A$, consideriamo due vettori spaziali ${}^A\mathbf{m} \in M^6$ e ${}^A\mathbf{f} \in F^6$ espressi rispetto alle coordinate del sistema di riferimento A . Desideriamo trovare i vettori che rappresentano $\dot{\mathbf{m}}$ e $\dot{\mathbf{f}}$ nelle coordinate A . Questi vettori sono dati dalle formule:

$$\begin{aligned} {}^A \left(\frac{d\mathbf{m}}{dt} \right) &= \frac{d({}^A\mathbf{m})}{dt} + {}^A\mathbf{v}_A \times {}^A\mathbf{m} \\ {}^A \left(\frac{d\mathbf{f}}{dt} \right) &= \frac{d({}^A\mathbf{f})}{dt} + {}^A\mathbf{v}_A \times^* {}^A\mathbf{f} \end{aligned}$$

Le espressioni sul lato sinistro denotano i vettori che rappresentano $\dot{\mathbf{m}}$ e $\dot{\mathbf{f}}$ nelle coordinate A , e le espressioni $\frac{d({}^A\mathbf{m})}{dt}$ e $\frac{d({}^A\mathbf{f})}{dt}$ denotano le derivate componente per componente di ${}^A\mathbf{m}$ e ${}^A\mathbf{f}$.

Definiamo quindi:

$${}^A\dot{\mathbf{v}} = {}^A \left(\frac{d\mathbf{v}}{dt} \right)$$

Definiamo la derivata componente per componente del vettore di coordinate ${}^A\mathbf{v}$:

$${}^A\dot{\mathbf{v}} = \frac{d({}^A\mathbf{v})}{dt}$$

Utilizzando questa notazione, le equazioni di differenziazione possono essere riscritte come:

$${}^A\dot{\mathbf{m}} = {}^A\dot{\mathbf{m}} + {}^A\mathbf{v}_A \times {}^A\mathbf{m}$$

$${}^A\dot{\mathbf{f}} = {}^A\dot{\mathbf{f}} + {}^A\mathbf{v}_A \times {}^A\mathbf{f}$$

1.10 Cambiamento di coordinate

Le trasformazioni di coordinate per i vettori spaziali richiedono due matrici di trasformazione: una per i vettori di moto e una per i vettori di forza.

- **Trasformazione di Moto:** Se $\mathbf{E}$ è la matrice di rotazione 3x3 che trasforma i vettori 3d da coordinate A a B e $\mathbf{r}$ è il vettore che esprime l'origine del sistema B in coordinate A, la trasformazione di coordinate di un vettore di moto, dal sistema di coordinate A al sistema di coordinate B, è:

$${}^B\mathbf{X}_A = \begin{bmatrix} \mathbf{E} & \mathbf{0} \\ -\mathbf{E}\mathbf{r}_{\times} & \mathbf{E} \end{bmatrix}$$

- **Trasformazione di Forza:** La trasformazione di coordinate per un vettore di forza è:

$${}^B\mathbf{X}_A^* = \begin{bmatrix} \mathbf{E} & -\mathbf{E}\mathbf{r}_{\times} \\ \mathbf{0} & \mathbf{E} \end{bmatrix}$$

Queste trasformazioni sono fondamentali per passare da un sistema di coordinate a un altro, mantenendo la coerenza delle descrizioni di moto e forza dei corpi rigidi.

Capitolo 2

Modellazione di Robot

Un robot è definito da un modello di sistema che descrive la configurazione e il movimento dei suoi componenti. I componenti fisici del robot che si muovono come unità rigide sono i **Corpi Rigidi (Segmenti)**. Ed il movimento relativo tra di essi è permesso dai **Giunti**, la cui dinamica è definita dai vettori $\mathbf{q}$, $\dot{\mathbf{q}}$ e $\ddot{\mathbf{q}}$ che rappresentano rispettivamente la posizione dei giunti del robot, la loro velocità e la loro accelerazione. Vengono di seguito definiti gli elementi del modello utilizzati nel corso della trattazione.

2.1 Topologia del sistema

La descrizione di come i differenti componenti del robot sono collegati tra loro è definita topologia di sistema e può essere espressa sotto forma di un grafico di connettività. In questa rappresentazione i nodi rappresentano i segmenti, ovvero le parti rigide del robot, e gli archi rappresentano le giunzioni che li connettono.

2.2 Albero Cinematico

Un albero cinematico è una struttura in cui ogni corpo rigido è collegato ad altri corpi rigidi tramite giunti, in modo che esista un solo percorso che collega una coppia di corpi e che, considerando i vari collegamenti, non si formino percorsi chiusi.

2.3 Vincoli di Movimento

Se il movimento di un corpo rigido è vincolato dai giunti, allora la sua velocità è un elemento di un sottospazio, $S \subseteq M^6$, chiamato il sottospazio di moto dei giunti.

Grado di libertà (di movimento):

$$\dim(S)$$

Grado di vincolo:

$$6 - \dim(S)$$

Il sottospazio S può essere rappresentato da una qualsiasi matrice $6 \times \dim(S)$ $\mathbf{S}$ che soddisfi

$$\text{Im}(\mathbf{S}) = S$$

2.4 Trasformazioni del Giunto (X_J)

Le matrici di cambio di coordinate, indicate con X_J , descrivono la relazione spaziale tra i corpi rigidi collegati da un giunto in funzione delle variabili di posizione del giunto stesso. Ogni giunto in un sistema di corpi rigidi consente movimenti relativi specifici tra i segmenti collegati, come rotazioni o traslazioni, e X_J rappresenta matematicamente queste trasformazioni.

La trasformazione del giunto è quindi una funzione delle variabili di posizione del giunto, che determinano l'esatta configurazione spaziale tra i due corpi collegati.

2.5 Trasformazioni dell'Albero (X_T)

Ogni trasformazione dell'albero $X_T(i)$ rappresenta la posizione del giunto i rispetto al corpo padre $\lambda(i)$ nel sistema.

Queste trasformazioni sono costanti e indipendenti dalle variabili di posizione del giunto.

2.6 Il robot ur5

Il modello del robot ur5 è descritto attraverso l'utilizzo di un file urdf [6], un formato standard di file di testo utilizzato per descrivere la struttura di un robot in termini di elementi fisici e cinematici

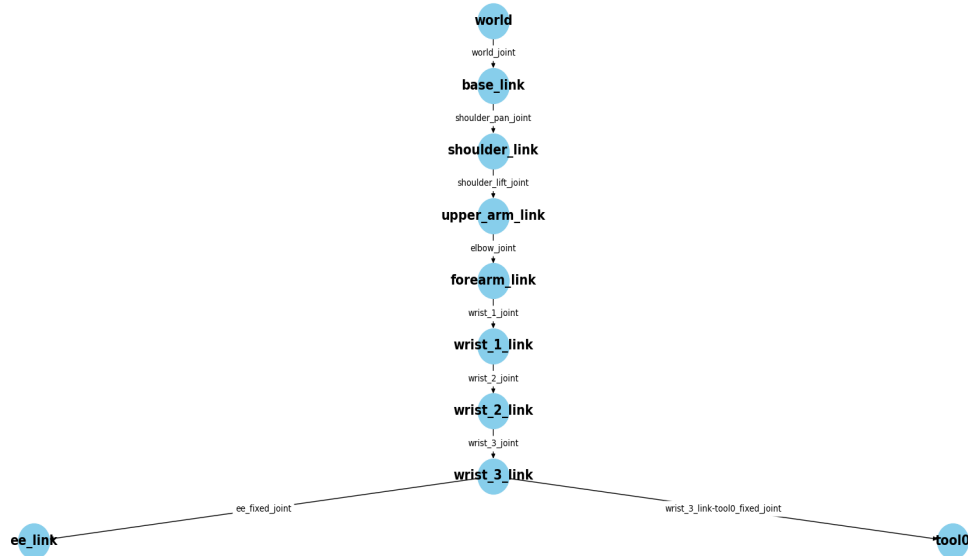


Figura 2.1: grafico di connettività del robot ur5

Da 2.1 si può osservare come il modello di ur5 è un albero cinematico, ciò permette di utilizzare gli algoritmi RNEA e CRBA, studiati nel prossimo capitolo, per la risoluzione della dinamica inversa del robot.

Verrà studiata la dinamica dei seguenti giunti:

- shoulder_pan_joint
- shoulder_lift_joint
- elbow_joint
- wrist_1_joint
- wrist_2_joint
- wrist_3_joint

Questo perché i giunti `wrist_3_linktool0_fixed_joint`, `ee_fixed_joint` e `world_joint` sono fissi e non permettono quindi nessun movimento.

Capitolo 3

Algoritmi per la dinamica inversa

Al fine di pianificare la traiettoria dei giunti del robot è necessario risolvere il problema della dinamica inversa ossia il calcolo della forza che deve essere applicata a un sistema per produrre in risposta $\mathbf{q}$, $\dot{\mathbf{q}}$ e $\ddot{\mathbf{q}}$ desiderati. Verranno quindi implementati degli algoritmi (ID) i quali utilizzeranno i dati nel modello di sistema per determinare la dinamica inversa del particolare sistema di corpo rigido descritto dal modello stesso.

$$\boldsymbol{\tau} = \text{ID}(\text{modello}, \mathbf{q}, \dot{\mathbf{q}}, \ddot{\mathbf{q}})$$

Dove $\boldsymbol{\tau}$ è il vettore delle forze applicate che agiscono sul sistema di corpo rigido

3.1 RNEA

L'algoritmo Ricorsivo di Newton-Eulero (RNEA) è fondamentale per il calcolo della dinamica inversa di sistemi robotici articolati. Questo algoritmo si distingue per la sua efficienza computazionale, calcolando le forze e i momenti necessari per produrre specifiche accelerazioni articolari in modo ricorsivo.

L'algoritmo RNEA si sviluppa in tre passi principali:

Calcolo delle Velocità e Accelerazioni

La velocità $\mathbf{v}_i$ e l'accelerazione $\mathbf{a}_i$ di ogni corpo nel sistema sono calcolate ricorsivamente. La velocità di un corpo è la somma della velocità del suo corpo genitore e della velocità attraverso il giunto che li collega:

$$\mathbf{v}_i = \mathbf{v}_{\lambda(i)} + \mathbf{S}_i \dot{\mathbf{q}}_i$$

dove $\mathbf{v}_{\lambda(i)}$ è la velocità del corpo genitore, $\mathbf{S}_i$ è la matrice del sottospazio di moto del giunto, e $\dot{\mathbf{q}}_i$ è la velocità articolare.

L'accelerazione è data da:

$$\mathbf{a}_i = \mathbf{a}_{\lambda(i)} + \mathbf{S}_i \ddot{\mathbf{q}}_i + \dot{\mathbf{S}}_i \dot{\mathbf{q}}_i$$

dove $\ddot{\mathbf{q}}_i$ è l'accelerazione articolare

Calcolo delle Forze sui Corpi

La forza netta $\mathbf{f}_i^B$ su ogni corpo è calcolata utilizzando l'equazione del moto:

$$\mathbf{f}_i^B = \mathbf{I}_i \mathbf{a}_i + \mathbf{v}_i \times^* \mathbf{I}_i \mathbf{v}_i$$

dove $\mathbf{I}_i$ è il tensore di inerzia del corpo, $\mathbf{a}_i$ è l'accelerazione del corpo, e $\mathbf{v}_i \times^* \mathbf{I}_i \mathbf{v}_i$ rappresenta le forze centrifughe e di Coriolis.

Calcolo delle Forze Trasmesse nei Giunti

Le forze trasmesse attraverso i giunti sono calcolate considerando le forze nette sui corpi:

$$\mathbf{f}_i = \mathbf{f}_i^B - \mathbf{f}_i^x + \sum_{j \in \mu(i)} \mathbf{f}_j$$

dove $\mathbf{f}_i^x$ rappresenta le forze esterne e $\sum_{j \in \mu(i)} \mathbf{f}_j$ la somma delle forze sui corpi figli.

Le forze generalizzate nei giunti sono poi determinate da:

$$\tau_i = \mathbf{S}_i^T \mathbf{f}_i$$

Implementazione e pseudo codice

L'algoritmo RNEA viene quindi implementato in due passaggi ricorsivi: un passaggio verso l'esterno, dal corpo padre ai corpi figli per calcolare velocità e accelerazioni, seguito da un passaggio verso l'interno per calcolare le forze articolari. Il seguente pseudocodice illustra l'algoritmo:

```

 $\mathbf{v}_0 = \mathbf{0}$ 
 $\mathbf{a}_0 = -\mathbf{a}_g$ 
for  $i = 1$  to  $N_B$  do
   $[\mathbf{X}_J, \mathbf{S}_i, \mathbf{v}_J, \mathbf{c}_J] = \text{jcalc}(\text{jtype}(i), \mathbf{q}_i, \dot{\mathbf{q}}_i)$ 
   ${}^i\mathbf{X}_{\lambda(i)} = \mathbf{X}_J \mathbf{X}_T(i)$ 
  if  $\lambda(i) \neq 0$  then
     ${}^i\mathbf{X}_0 = {}^i\mathbf{X}_{\lambda(i)} {}^{\lambda(i)}\mathbf{X}_0$ 
  end
   $\mathbf{v}_i = {}^i\mathbf{X}_{\lambda(i)} \mathbf{v}_{\lambda(i)} + \mathbf{v}_J$ 
   $\mathbf{a}_i = {}^i\mathbf{X}_{\lambda(i)} \mathbf{a}_{\lambda(i)} + \mathbf{S}_i \ddot{\mathbf{q}}_i + \mathbf{c}_J + \mathbf{v}_i \times \mathbf{v}_J$ 
   $\mathbf{f}_i = \mathbf{I}_i \mathbf{a}_i + \mathbf{v}_i \times^* \mathbf{I}_i \mathbf{v}_i - {}^i\mathbf{X}_0^* \mathbf{f}_i^x$ 
end
for  $i = N_B$  to 1 do
   $\tau_i = \mathbf{S}_i^T \mathbf{f}_i$ 
  if  $\lambda(i) \neq 0$  then
     $\mathbf{f}_{\lambda(i)} = \mathbf{f}_{\lambda(i)} + {}^{\lambda(i)}\mathbf{X}_i^* \mathbf{f}_i$ 
  end
end
end

```

Con

$$\begin{aligned} \boldsymbol{v}_{Ji} &= \boldsymbol{S}_i \dot{\boldsymbol{q}}_i \\ \boldsymbol{c}_{Ji} &= \dot{\boldsymbol{S}}_i \dot{\boldsymbol{q}}_i \end{aligned}$$

3.2 CRBA

La dinamica di un sistema di corpi rigidi può essere descritta dall'equazione del moto:

$$\boldsymbol{\tau} = \mathbf{H}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) \quad (3.1)$$

dove:

- $\boldsymbol{\tau}$ è il vettore delle forze generalizzate applicate;
- $\mathbf{H}$ è la matrice di inerzia del sistema (detta anche matrice di massa $\mathbf{M}$);
- $\ddot{\mathbf{q}}$ è il vettore delle accelerazioni dei giunti;
- $\mathbf{C}$ rappresenta le forze di Coriolis, centrifughe e di gravità.

Al fine di risolvere la dinamica inversa è possibile ricavare $\mathbf{C}$ dall'algoritmo RNEA. L'idea generale è quella di eseguire l'algoritmo RNEA senza considerare l'accelerazione $\ddot{\mathbf{q}}$. In questo modo, il termine risultante sarà solo funzione delle velocità $\dot{\mathbf{q}}$ e della configurazione $\mathbf{q}$ del sistema, ottenendo quindi $\mathbf{C}$. Per risolvere la dinamica inversa è inoltre necessario calcolare la matrice $\mathbf{H}$, che verrà ottenuta tramite CRBA.

Matrice di Inerzia del sistema $\mathbf{H}$

$\mathbf{H}$ è una matrice $n \times n$ simmetrica e definita positiva, organizzata in un array $N_B \times N_B$ di sottomatrici (n indica il numero totale di gradi di libertà nel sistema e N_B il numero di corpi). Se n_i è il grado di libertà del giunto i , allora $\mathbf{H}_{ij}$ è la sottomatrice $n_i \times n_j$ di $\mathbf{H}$ che occupa la riga i e la colonna j del blocco. Fisicamente, $\mathbf{H}_{ij}$ mette in relazione l'accelerazione al giunto j con la forza al giunto i . Se ogni giunto nell'albero ha solo un grado di libertà, allora $n = N_B$, e $\mathbf{H}_{ij}$ è la matrice 1×1 sulla riga i e colonna j di $\mathbf{H}$.

Descrizione dell'Algoritmo

Il CRBA procede calcolando per ciascun corpo l'inerzia composta dell'insieme dei corpi supportati da quel corpo stesso, utilizzando una formula ricorsiva che somma le inerzie dei singoli corpi. Una volta ottenute le inerzie composite, la matrice $\mathbf{H}$ viene determinata combinando queste informazioni, in modo da considerare l'influenza di ciascun corpo sull'intero sistema.

```

 $H = 0$ 
for  $i = 1$  to  $N_B$  do
   $I_i^c = I_i$ 
end
for  $i = N_B$  to 1 do
  if  $\lambda(i) \neq 0$  then
     $I_{\lambda(i)}^c = I_{\lambda(i)}^c + {}^{\lambda(i)}X_i^* I_i^c {}^iX_{\lambda(i)}$ 
  end
   $F = I_i^c S_i$ 
   $H_{ii} = S_i^T F$ 
   $j = i$ 
  while  $\lambda(j) \neq 0$  do
     $F = {}^{\lambda(j)}X_j^* F$ 
     $j = \lambda(j)$ 
     $H_{ij} = F^T S_j$ 
     $H_{ji} = H_{ij}^T$ 
  end
end
end

```

3.3 Implementazione degli algoritmi con libreria ADAM

Gli algoritmi per la dinamica esposti precedentemente sono implementati nella libreria ADAM [6]. Tuttavia mentre l'algoritmo CRBA è completamente implementato all'interno della libreria non è così per RNEA. La versione ampliata dell'algoritmo è riportata su git-hub [4]. È stato proposto l'inserimento di tale modifica nella versione ufficiale della libreria.

Implementazione di RNEA in ADAM

La libreria ADAM fornisce una versione ridotta dell'algoritmo RNEA, specificamente progettata per il calcolo delle forze di bias. Questa versione dell'algoritmo è finalizzata a determinare le forze dovute ai termini di Coriolis, centrifughi e gravitazionali, escludendo le accelerazioni articolari $\ddot{\mathbf{q}}$, riducendo così la complessità computazionale. Di conseguenza, il risultato ottenuto non include il contributo della matrice di inerzia $\mathbf{H}(\mathbf{q})$ moltiplicato per $\ddot{\mathbf{q}}$. Il focus dell'algoritmo sviluppato in ADAM è quindi sul calcolo della matrice delle forze di Coriolis e centrifughe, rappresentata da $\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$.

Modifiche per l'Implementazione Completa di RNEA

Per ottenere una versione completa e accurata dell'algoritmo RNEA, capace di calcolare esattamente le forze richieste per un movimento desiderato, è stato necessario apportare modifiche all'implementazione di ADAM:

Una delle modifiche principali è stata l'introduzione del calcolo delle accelerazioni articolari $\ddot{\mathbf{q}}$ all'interno dell'espressione:

$$\mathbf{a}_i = {}^i\mathbf{X}_{\lambda(i)}\mathbf{a}_{\lambda(i)} + \mathbf{S}_i\ddot{\mathbf{q}}_i + \mathbf{c}_J + \mathbf{v}_i \times \mathbf{v}_J$$

In quanto in ADAM non era considerata. Questo ha permesso di includere nella formulazione le forze necessarie per accelerare le giunture del robot. Ciò garantisce che l'algoritmo possa calcolare tutte le componenti di $\boldsymbol{\tau} = \mathbf{H}(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})$.

Capitolo 4

Problema di ottimizzazione della traiettoria

4.1 Introduzione

Nel contesto della robotica e della mecatronica, il controllo ottimo è una tecnica fondamentale utilizzata per determinare le traiettorie e i segnali di controllo che minimizzano una funzione di costo, garantendo nel contempo il rispetto delle dinamiche del sistema e di eventuali vincoli. Questo capitolo introduce il problema di controllo ottimo applicato a un robot articolato, con particolare attenzione alla formulazione e alla risoluzione del problema tramite l'utilizzo della libreria CasADi [2] e ADAM [6]. Il problema considerato consiste nel determinare una traiettoria ottimale per un robot manipolatore, con l'obiettivo di raggiungere una configurazione desiderata $\mathbf{q}_{\text{des}}$ a partire da una configurazione iniziale $\mathbf{q}_0$, minimizzando una funzione di costo che penalizza deviazioni dalla configurazione desiderata, oltre a velocità e accelerazioni elevate. Si procede a descrivere la formulazione matematica del problema.

4.2 Variabili del Problema

Le variabili di stato del problema sono:

- $\mathbf{q}_t$: Vettore delle posizioni articolari al tempo t .
- $\dot{\mathbf{q}}_t$: Vettore delle velocità articolari al tempo t .

$\ddot{\mathbf{q}}_t$ è la variabile di controllo del problema, ovvero i parametri da ottimizzare.

4.3 Equazioni di Stato

Le equazioni di stato descrivono la dinamica del sistema e rappresentano i vincoli differenziali discretizzati che collegano le variabili del problema. Le equazioni sono date da:

$$\mathbf{q}_{t+1} = \mathbf{q}_t + \Delta t \cdot \dot{\mathbf{q}}_t$$

$$\dot{\mathbf{q}}_{t+1} = \dot{\mathbf{q}}_t + \Delta t \cdot \ddot{\mathbf{q}}_t$$

Dove Δt rappresenta l'intervallo di tempo discreto scelto per la discretizzazione del problema.

4.4 Funzione di Costo

La funzione di costo da minimizzare è formulata come segue:

$$J = \sum_{k=0}^{N-1} (\| \mathbf{q}_t - \mathbf{q}_{\text{des}} \|^2 + w_v \| \dot{\mathbf{q}}_t \|^2 + w_a \| \ddot{\mathbf{q}}_t \|^2) + w_f \| \mathbf{q}_{t=tf} - \mathbf{q}_{\text{des}} \|^2 + w_f \| \dot{\mathbf{q}}_{t=tf} \|^2$$

Dove:

- $N \cdot \Delta t$ è l'orizzonte temporale del problema
- $t = k \cdot \Delta t$ e $tf = N \cdot \Delta t$
- $\| \mathbf{q}_t - \mathbf{q}_{\text{des}} \|^2$ penalizza la deviazione della posizione articolare corrente $\mathbf{q}_t$ dalla posizione desiderata $\mathbf{q}_{\text{des}}$.
- $w_v \| \dot{\mathbf{q}}_t \|^2$ penalizza le velocità elevate, favorendo traiettorie più morbide.
- $w_a \| \ddot{\mathbf{q}}_t \|^2$ penalizza le accelerazioni elevate, che si riflettono in richieste di coppie più alte.
- $w_f \| \mathbf{q}_{t=tf} - \mathbf{q}_{\text{des}} \|^2 + w_f \| \dot{\mathbf{q}}_{t=tf} \|^2$ penalizza la deviazione della configurazione finale da quella desiderata e velocità finali elevate

I pesi scalari w_v , w_a e w_f permettono di bilanciare l'importanza relativa tra i diversi termini della funzione di costo.

4.5 Vincoli del Problema

Limiti di coppia

Per garantire che il robot operi entro i limiti di coppia specificati, si impongono i seguenti vincoli sulle coppie articolari:

$$\tau_{\min} \leq \text{DI}(\mathbf{q}_t, \dot{\mathbf{q}}_t, \ddot{\mathbf{q}}_t) \leq \tau_{\max}$$

Dove $\tau_{\min}$ e $\tau_{\max}$ rappresentano rispettivamente i limiti inferiori e superiori delle coppie articolari per il robot considerato.

Nome del giunto	$\tau_{\min}$ (Nm)	$\tau_{\max}$ (Nm)
shoulder_pan_joint	-150.0	150.0
shoulder_lift_joint	-150.0	150.0
elbow_joint	-150.0	150.0
wrist_1_joint	-28.0	28.0
wrist_2_joint	-28.0	28.0
wrist_3_joint	-28.0	28.0

Tabella 4.1: Limiti di sforzo ($\tau_{\min}$ e $\tau_{\max}$) per i diversi giunti

La τ può essere ricavata attraverso metodi differenti e scopo di questo problema è vedere quale degli approcci provochi un tempo minori di risoluzione del problema, in quanto sono i vincoli della dinamica il fattore che influenza maggiormente i tempi di risoluzione del problema

Approccio 1: Dinamica Inversa con CRBA e RNEA

In questo approccio, la dinamica inversa del robot è calcolata utilizzando due algoritmi chiave:

- **CRBA (Composite Rigid Body Algorithm):** Questo algoritmo calcola la matrice di inerzia $M(q)$ del robot in modo efficiente.
- **RNEA (Recursive Newton-Euler Algorithm):** Questo algoritmo, ponendo $\ddot{q} = \mathbf{0}$ calcola il vettore dei termini non lineari $C(q, \dot{q})$ che include gli effetti delle forze centripete, di Coriolis e gravitazionali.

La dinamica inversa è quindi espressa come:

$$\tau = M(q)\ddot{q} + C(q, \dot{q})$$

Dove τ rappresenta il vettore delle coppie articolari necessarie per ottenere l'accelerazione desiderata $\ddot{q}$.

Approccio 2: Dinamica Inversa Diretta con RNEA

In questo secondo approccio, si utilizza direttamente l'algoritmo RNEA per calcolare le coppie articolari τ , bypassando il calcolo esplicito della matrice di inerzia:

$$\tau = \text{RNEA}(q, \dot{q}, \ddot{q})$$

Limiti di configurazione

I giunti del robot sono limitati nelle configurazioni che possono assumere

Nome del giunto	Limite Inferiore (rad)	Limite superiore (rad)
shoulder_pan_joint	-6.28318530718	6.28318530718
shoulder_lift_joint	-6.28318530718	6.28318530718
elbow_joint	-3.14159265359	3.14159265359
wrist_1_joint	-6.28318530718	6.28318530718
wrist_2_joint	-6.28318530718	6.28318530718
wrist_3_joint	-6.28318530718	6.28318530718

Tabella 4.2: Limiti di posizione per i differenti giunti

4.6 Risoluzione del Problema con CasADi

Per risolvere il problema di controllo ottimo formulato, si utilizza la libreria CasADi, che è particolarmente adatta per la modellazione e la risoluzione di problemi di ottimizzazione non lineare, come mostrato nell'articolo "CasADi: a software framework for nonlinear optimization and optimal control" [1].

Il problema è stato discretizzato nel tempo, le configurazioni dei giunti e i vincoli sulle coppie sono stati inclusi come vincoli del problema di ottimizzazione. La funzione di costo è stata definita come obiettivo da minimizzare, e il problema è stato risolto per ottenere le traiettorie ottimali per le variabili di controllo $\ddot{q}_t$, e trovare quindi attraverso le relazioni espresse in precedenza le variabili di stato q_t e $\dot{q}_t$. L'implementazione del codice per la risoluzione del problema è caricata su git-hub [4].

4.7 Confronto tra gli Approcci

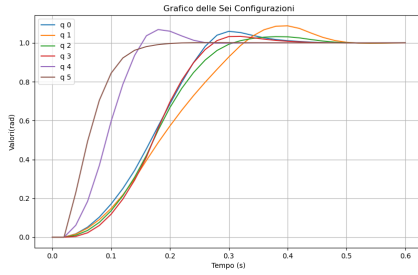
Vengono ora presentati i risultati ottenuti, con i due approcci descritti, per differenti configurazioni del problema di ottimizzazione.

Problema 1

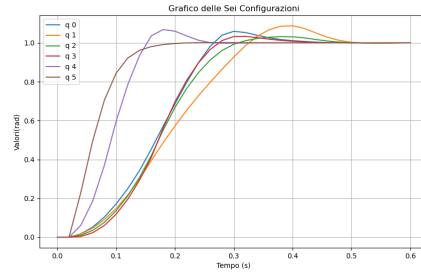
I valori scelti per la variabili del problema sono:

- $\Delta t = 0,02$
- $N = 30$
- $\mathbf{q}_0 = (0, 0, 0, 0, 0, 0)$
- $\dot{\mathbf{q}}_0 = (0, 0, 0, 0, 0, 0)$
- $\mathbf{q}_{\text{des}} = (1, 1, 1, 1, 1, 1)$
- $w_v = 1 \times 10^{-4}$
- $w_a = 1 \times 10^{-6}$
- $w_f = 1 \times 10^2$

Il risultato è espresso in 4.1 ed i tempi di esecuzione sono di 7.46 secondi per l'approccio con RNEA e di 15.87 secondi per l'approccio con CRBA+RNEA



(a) RNEA



(b) CRBA+RNEA

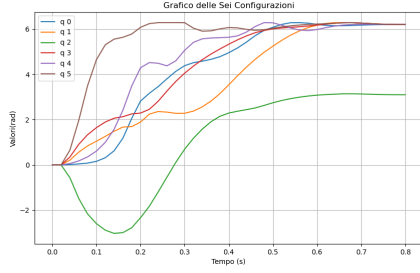
Figura 4.1: Confronto tra i due approcci

Problema 2

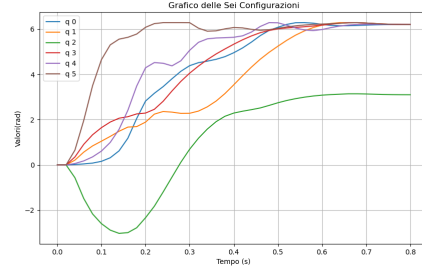
I valori scelti per la variabili del problema, al fine di portare i giunti alla configurazione limite, sono:

- $\Delta t = 0,02$
- $N = 40$
- $\mathbf{q}_0 = (0, 0, 0, 0, 0, 0)$
- $\dot{\mathbf{q}}_0 = (0, 0, 0, 0, 0, 0)$

- $\mathbf{q}_{\text{des}} = (6.2, 6.2, 3.1, 6.2, 6.2, 6.2)$
- $w_v = 1 \times 10^{-4}$
- $w_a = 1 \times 10^{-6}$
- $w_f = 1 \times 10^2$



(a) RNEA



(b) CRBA+RNEA

Figura 4.2: Confronto tra i due Approcci

Il risultato è espresso in 4.2 ed i tempi di esecuzione sono di 33.61 secondi per l'approccio con RNEA e di 54.71 secondi per l'approccio con CRBA+RNEA

Risultati comparazione

Una volta risolto il problema utilizzando entrambi gli approcci descritti, si è proceduto a confrontare i risultati in termini di tempo di calcolo e qualità della traiettoria ottenuta. I risultati ottenuti dimostrano che le traiettorie dei giunti ottenute tramite i due approcci sono le stesse. L'approccio 2, che utilizza solo l'algoritmo RNEA, offre un vantaggio in termini di tempi di calcolo, rendendolo preferibile in contesti dove l'efficienza computazionale è critica.

Conclusioni

Questa tesi ha esplorato e confrontato due approcci distinti per l'ottimizzazione della traiettoria di un robot manipolatore, con un focus particolare sull'efficienza computazionale. È stato necessario lo studio e la modifica della libreria ADAM al fine di implementare CRBA e RNEA. Successivamente attraverso un'analisi dettagliata dei due metodi di ottimizzazione, si è dimostrato che l'approccio basato esclusivamente sull'algoritmo Ricorsivo di Newton-Eulero (RNEA) offre un significativo vantaggio in termini di tempo di calcolo rispetto a un approccio combinato che include anche l'Algoritmo del Corpo Rigido Composito (CRBA).

I risultati mostrano che, pur mantenendo la stessa qualità delle traiettorie, l'approccio RNEA risulta più efficiente, rendendolo preferibile in scenari dove la rapidità di calcolo è essenziale.

Il risultato è quindi compatibile con quanto atteso teoricamente.

Bibliografia

- [1] Joel A. E. Andersson et al. «CasADi: a software framework for nonlinear optimization and optimal control». In: *Mathematical Programming Computation* 11 (2019), pp. 1–36. DOI: 10.1007/s12532-018-0139-4.
- [2] CasADi Developers. *CasADi: a software framework for nonlinear optimization and optimal control*. <https://github.com/casadi/casadi>. Versione 3.5.5. 2024.
- [3] Roy Featherstone. *Rigid Body Dynamics Algorithms*. New York, NY: Springer, 2008. ISBN: 978-0-387-74314-1. DOI: 10.1007/978-1-4899-7560-7.
- [4] Giacomo Montagna. *Controllo Ottimo della Traiettoria di Robot*. <https://github.com/Giacco02/Controllo-Ottimo-della-Traiettoria-di-Robot>. Accessed: 2024-08-18. 2024.
- [5] Bruno Siciliano e Oussama Khatib, cur. *Springer Handbook of Robotics*. Berlin, Heidelberg: Springer, 2008. ISBN: 978-3-540-23957-4. DOI: 10.1007/978-3-540-30301-5.
- [6] Fondazione Istituto Italiano di Tecnologia. *ADAM: The Advanced Dynamic Autonomous Manipulator*. <https://github.com/ami-iit/adam>. Accessed: 2024-08-18. 2024.

Elenco delle figure

2.1	grafico di connettività del robot ur5	7
4.1	Confronto tra i due approcci	18
4.2	Confronto tra i due Approcci	19

Elenco delle tabelle

4.1	Limiti di sforzo ($\tau_{\min}$ e $\tau_{\max}$) per i diversi giunti	16
4.2	Limiti di posizione per i differenti giunti	17

Ringraziamenti

Vorrei esprimere la mia sincera gratitudine al Prof. Andrea Del Prete per la sua preziosa guida e supporto durante la stesura di questa tesi.

Un ringraziamento speciale va alla mia famiglia, per il loro amore e sostegno incondizionato. Grazie per aver sempre creduto in me.

Infine, grazie ai miei amici, per il loro incoraggiamento e la loro presenza costante. Senza di voi, questo percorso sarebbe stato molto più difficile.

A tutti voi, il mio più profondo grazie.